

What is React.js? How is it different from other JavaScript frameworks and libraries?

-> React.js is a popular JavaScript library used for building user interfaces, especially for single-page applications.

-> It allows developers to create reusable UI components and efficiently update the user interface when data changes. Unlike traditional frameworks like Angular, React is a library focused only on the view layer.

-> It uses a component-based architecture and gives developers flexibility to choose other tools for routing, state management, etc.

Explain the core principles of React such as the virtual DOM and component-based architecture.

React is built on key principles:

1. Virtual DOM: React creates a lightweight copy of the real DOM in memory. When state changes, React updates the virtual DOM first, compares it with the previous version, and then updates only the changed parts in the real DOM. This improves performance.

2. Component-Based Architecture: UI is divided into small, reusable components. Each component manages its own logic and can be reused across the application, making code more maintainable.

What are the advantages of using React.js in web development?

- Fast performance due to Virtual DOM - Reusable components
- Easy to learn and use
- Strong community support
- SEO-friendly with server-side rendering
- Flexibility to integrate with other libraries

What is JSX in React.js? Why is it used?

-> JSX (JavaScript XML) is a syntax extension for JavaScript used in React.

-> It allows developers to write HTML-like code inside JavaScript.

-> JSX makes code more readable and easier to write UI components.

-> It is used because it simplifies the process of creating and structuring UI elements in React applications.

How is JSX different from regular JavaScript? Can you write JavaScript inside JSX?

-> JSX is different from regular JavaScript because it looks similar to HTML and allows embedding UI structure directly within JavaScript code.

-> Yes, you can write JavaScript inside JSX using curly braces {}. Inside these braces, you can include expressions such as variables, functions, and calculations.

Discuss the importance of using curly braces {} in JSX expressions.

-> Curly braces {} in JSX are used to embed JavaScript expressions inside JSX code.

-> They allow dynamic content rendering, such as displaying variables, performing calculations, calling functions, and conditional rendering. Without curly braces, JSX would not be able to execute JavaScript logic, making it less dynamic and flexible.

What are components in React? Explain the difference between functional components and class components.

-> Components in React are reusable, independent building blocks of a user interface.

-> Each component represents a part of the UI and can manage its own logic and state.

Functional Components:

- Simple JavaScript functions
- Use hooks for state and lifecycle
- Easier to read and write
- Preferred in modern React

Class Components: -

- ES6 classes that extend React.Component
- Use render() method to return JSX
- Have lifecycle methods - More complex compared to functional components

How do you pass data to a component using props?

-> Props are used to pass data from a parent component to a child component.

-> They are passed as attributes in JSX and accessed inside the component.

Example:

`<ChildComponent name="John" />` Inside ChildComponent: `props.name`

What is the role of render() in class components?

- > The render() method is used in class components to describe what should be displayed on the UI.
- > It returns JSX that represents the component's UI. Whenever the state or props change, the render() method is called again to update the UI.

What are props in React.js? How are props different from state?

- Props are used to pass data from a parent component to a child component in React.
- They are read-only and cannot be modified by the child component.

Difference between Props and State: -

- > Props are immutable (cannot be changed), while state is mutable.
- > Props are passed from parent to child, whereas state is managed within the component.
- > Props are used for communication between components, while state is used for managing internal data.

Explain the concept of state in React and how it is used to manage component data.

- > State in React is an object that stores data specific to a component.
- > It is used to control the behavior and rendering of that component.
- > When the state changes, React re-renders the component to reflect the updated data.
- > State is commonly used for handling user input, API data, toggles, and dynamic UI updates.

Why is this.setState() used in class components, and how does it work?

- > this.setState() is used in class components to update the component's state.
- > It schedules an update to the state and tells React to re-render the component. Instead of directly modifying the state, setState() ensures that updates are handled efficiently and asynchronously.
- > It merges the new state with the existing state and triggers a re-render of the component.

How are events handled in React compared to vanilla JavaScript? Explain the concept of synthetic events.

-> In React, events are handled using camelCase naming (e.g., onClick) instead of lowercase (onclick) used in vanilla JavaScript.

-> Instead of adding event listeners manually, React uses event handler functions directly in JSX.

-> React uses Synthetic Events, which are wrapper objects around native browser events. They provide a consistent API across different browsers and improve performance by reusing event objects.

What are some common event handlers in React.js? Provide examples of onClick, onChange, and onSubmit.

Common event handlers include: -

- **onClick:**

Example: `<button onClick={handleClick}>Click Me</button>`

- **onChange:**

Example: `<input type="text" onChange={handleChange} />`

- **onSubmit:**

Example: `<form onSubmit={handleSubmit}>...</form>`

Why do you need to bind event handlers in class components?

-> In class components, event handler methods do not automatically bind to the class instance.

-> Binding is required to ensure that 'this' refers to the correct component instance. Without binding, 'this' may be undefined inside the handler. Binding can be done in the constructor or by using arrow functions.

What is conditional rendering in React? How can you conditionally render elements in a React component?

-> Conditional rendering in React means displaying different UI elements or components based on certain conditions or states.

-> It works similarly to conditional statements in JavaScript. You can conditionally render elements using: - if-else statements - ternary operators - logical AND (&&) operator This allows developers to dynamically update the UI based on user actions, data, or

application state.

Explain how if-else, ternary operators, and && (logical AND) are used in JSX for conditional rendering

1. if-else:

Used outside JSX to decide what to render.

Example:

```
if (isLoggedIn) {  
  return <h1>Welcome</h1>;  
} else {  
  return <h1>Please Login</h1>;  
}
```

2. Ternary Operator:

Used inside JSX for inline conditions.

Example: {isLoggedIn ? <h1>Welcome</h1> : <h1>Please Login</h1>}

3. Logical AND (&&):

Used to render something only if the condition is true.

Example: {isLoggedIn && <h1>Welcome</h1>} If the condition is false, nothing is rendered.

How do you render a list of items in React? Why is it important to use keys when rendering lists?

-> In React, lists are rendered using the map() function. This function allows you to iterate over an array and return JSX elements for each item.

Example: const items = ['Apple', 'Banana', 'Orange'];

```
items.map((item, index) => <li key={index}>{item}</li>);
```

-> Keys are important because they help React identify which items have changed, been added, or removed. This improves performance and ensures efficient updates to the UI.

What are keys in React, and what happens if you do not provide a unique key?

-> Keys are unique identifiers assigned to elements in a list.

->They help React track each element during re-rendering.

If you do not provide a unique key:

- React may show warnings in the console
- UI updates may become inefficient
- It can lead to incorrect rendering or bugs when items are added, removed, or reordered It is recommended to use a unique value like an ID instead of the array index whenever possible.

How do you handle forms in React? Explain the concept of controlled components.

- Forms in React are handled using components that manage form data through state.
- A controlled component is a form element whose value is controlled by React state.

In controlled components: -

- Form data is handled by the component's state
- Input values are set using state
- Changes are handled using event handlers like onChange

Example:

```
<input type="text" value={this.state.name}  
onChange={this.handleChange} />
```

This ensures that React has full control over the form data.

What is the difference between controlled and uncontrolled components in React?

Controlled Components: -

- Managed by React state
- Data flows through state
- Easier validation and control

Uncontrolled Components: -

- Managed by the DOM itself
- Use refs to access values
- Less control compared to controlled components Controlled components are preferred because they provide better control, validation, and predictability.

What are lifecycle methods in React class components? Describe the phases of a component's lifecycle.

-> Lifecycle methods are special methods in React class components that are automatically called at different stages of a component's life.

-> They allow developers to control what happens when a component is created, updated, or destroyed.

Phases of Lifecycle:

1. Mounting Phase:

- When the component is created and inserted into the DOM.
- Methods: `constructor()`, `render()`, `componentDidMount()`

2. Updating Phase:

- When the component's state or props change
- Methods: `render()`, `componentDidUpdate()`

3. Unmounting Phase:

- When the component is removed from the DOM
- Method: `componentWillUnmount()`

Explain the purpose of `componentDidMount()`, `componentDidUpdate()`, and `componentWillUnmount()`.

- **`componentDidMount()`**: Called after the component is rendered for the first time. Used for API calls, subscriptions, or initializing data.

- **`componentDidUpdate()`**: Called after the component updates due to changes in state or props. Used to perform actions based on updated data.

- **`componentWillUnmount()`**: Called just before the component is removed from the DOM. Used for cleanup like cancelling API calls, removing event listeners, or clearing timers.

What are React hooks? How do `useState()` and `useEffect()` hooks work in functional components?

React Hooks are functions that allow you to use state and lifecycle features in functional components.

- **`useState()`**: Used to declare state in a functional component.

Example: `const [count, setCount] = useState(0);`

- **`useEffect()`**: Used to perform side effects such as API calls, subscriptions, or DOM updates.

It runs after the component renders.

What problems did hooks solve in React development? Why are hooks considered an important addition to React?

Hooks solved problems like:

- Complex class components
- Difficult code reuse
- Confusing lifecycle methods

They are important because:

- Simpler and cleaner code
- Better logic reuse with custom hooks
- No need for class components

What is useReducer? How we use in react app?

-useReducer is a hook used for managing complex state logic. It works similar to Redux.

-Example:

```
const [state, dispatch] = useReducer(reducer, initialState);
```

-It is useful when state depends on previous state or has multiple sub-values.

What is the purpose of useCallback & useMemo Hooks?

- **useCallback**: Returns a memoized function to prevent unnecessary re-creation of functions.

- **useMemo**: Returns a memoized value to avoid expensive calculations on every render.

What's the Difference between the useCallback & useMemo Hooks?

- useCallback memoizes functions

- useMemo memoizes values `useCallback(fn, deps) === useMemo(() => fn, deps)`

What is useRef? How to work in react app?

useRef is a hook used to persist values across renders without causing re-renders.

It is used for: -

- Accessing DOM elements
- Storing mutable values

Example: `const inputRef = useRef(null);`

What is React Router? How does it handle routing in single-page applications?

-> React Router is a library used in React applications to enable navigation between different components or pages without reloading the entire page.

-> In single-page applications (SPAs), React Router updates the URL and renders the appropriate component dynamically using the browser's history API, without making full page requests to the server.

Explain the difference between BrowserRouter, Route, Link, and Switch components in React Router.

- **BrowserRouter**: It is the main router component that uses the HTML5 history API to keep the UI in sync with the URL.

- **Route**: Used to define a mapping between a URL path and a component.

Example:

```
<Route path="/home" component={Home} />
```

- **Link**: Used to navigate between routes without reloading the page.

Example: `<Link to="/home">Go to Home</Link>`

- **Switch**: Used to render only the first matching Route among multiple routes.

What do you mean by RESTful web services?

-> RESTful web services are APIs that follow REST (Representational State Transfer) principles.

-> They use HTTP methods like GET, POST, PUT, and DELETE to perform operations on resources.

-> Each resource is identified by a URL, and data is usually exchanged in JSON format.

What is Json-Server? How we use in React?

-> Json-Server is a lightweight tool that allows you to create a fake REST API using a JSON file.

-> It is useful for frontend development and testing without a real backend. In React, you can run Json-Server and make API calls (GET, POST, etc.) to fetch or send data.

How do you fetch data from a Json-server API in React? Explain the role of fetch() or axios() in making API requests.

-> You can fetch data using fetch() or axios:

- **fetch()**: Built-in JavaScript method for making HTTP requests.

Example:

```
fetch('http://localhost:3000/users')  
.then(res => res.json())  
.then(data => console.log(data));
```

- **axios()**: A popular library for making HTTP requests with better features.

Example:

```
axios.get('http://localhost:3000/users')  
.then(res => console.log(res.data));
```

Both are used to communicate with APIs and retrieve or send data.

What is Firebase? What features does Firebase offer?

Firebase is a platform by Google for building web and mobile applications.

Features include:

- Realtime Database & Firestore
- Authentication
- Hosting
- Cloud Functions
- Storage
- Analytics

Discuss the importance of handling errors and loading states when working with APIs in React.

Handling errors and loading states is important to improve user experience.

- **Loading state**: Shows a loader or message while data is being fetched.

- **Error handling**: Displays error messages if something goes wrong. This ensures better usability, prevents crashes, and provides feedback to users.

What is the Context API in React? How is it used to manage global state across multiple components?

-> The Context API in React is a feature that allows you to share data (global state) across multiple components without passing props manually at every level (prop drilling).

-> It is used to manage global data such as user authentication, themes, or settings. By creating a context, you can provide data at a higher level and access it in any nested component.

Explain how createContext() and useContext() are used in React for sharing state.

- **createContext():** Used to create a new context object.

Example:

```
const MyContext = React.createContext();
```

- **Provider:** Wraps components and provides the context value.

Example:

```
<MyContext.Provider value={data}>...</MyContext.Provider>
```

- **useContext():** A hook used to access the context value inside functional components.

Example: `const value = useContext(MyContext);`

This allows components to directly consume shared data without passing props manually.

What is Redux, and why is it used in React applications? Explain the core concepts of actions, reducers, and the store.

-> Redux is a state management library used in React applications to manage global state in a predictable way.

-> It is especially useful for large applications where state needs to be shared across multiple components.

Core Concepts:

- **Actions:** Actions are plain JavaScript objects that describe what happened. They have a `type` property and may include additional data.

- **Reducers:** Reducers are functions that take the current state and an action as input, and return a new state. They define how the state changes in response to actions.

- **Store:** The store is a central place where the application state is stored. It allows access to state, dispatching actions, and registering listeners.

How does Recoil simplify state management in React compared to Redux?

-> Recoil is a state management library for React that provides a simpler and more flexible way to manage state.

Differences and Advantages:

- Less boilerplate code compared to Redux
- No need for actions and reducers
- Uses atoms (state units) and selectors (derived state)
- Easier to understand and implement
- Better integration with React hooks Recoil makes state management more intuitive and reduces complexity, especially for smaller or medium-sized applications.